Cloud & DevOps Daily

How to Deploy Code Without Downtime (Zero-Downtime Deploys)

Tuesday, September 01, 2026

Zero Downtime Deployments: A Beginner's Practical Guide

Zero-downtime deployment allows you to update web applications without disconnecting active users. Mastering this core DevOps practice ensures high availability and keeps your app online during updates.

What you'll learn:

1	1. Kubernetes Rolling Updates
2	2. Setting Up Readiness Probes
3	3. Instant Switch with Blue-Green Deploys
4	4. Graceful Shutdown in Application Code

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Kubernetes Rolling Updates

A Rolling Update gradually replaces instances of your old application with new ones. Kubernetes updates pods incrementally, ensuring enough healthy pods remain active to handle incoming user traffic. If a new pod fails to start, the deployment automatically halts to protect the application.

```
$ kubectl set image deployment/my-web-app web-container=my-web-app:v2.0 --record
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Setting Up Readiness Probes

Readiness probes check if your container is ready to serve traffic after booting. Without them, Kubernetes sends user requests to an app that is still loading files or connecting to databases, leading to instant 500 errors. Always expose a simple health check endpoint in your code.

```
$ readinessProbe:
    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Instant Switch with Blue-Green Deploys

In a Blue-Green setup, you run two identical environments: Blue (current live) and Green (new release). You deploy and test code on Green without affecting real users. Once verified, you change your reverse proxy configuration to route all live traffic to Green instantly.

```
$# Test and reload Nginx configuration to point to new upstream
 sudo nginx -t && sudo nginx -s reload
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Graceful Shutdown in Application Code

When a container receives a stop signal (SIGTERM), it shouldn't die instantly. A graceful shutdown handler stops accepting new incoming connections, finishes processing existing user requests, closes database connection pools, and exits cleanly within a set grace period.

```
$ process.on('SIGTERM', () => {
   console.log('Closing HTTP server...');
   server.close(() => {
     console.log('HTTP server closed cleanly.');
     process.exit(0);
   });
 });
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always configure Liveness and Readiness probes in your container manifests.
- + Keep database schema changes backward-compatible so old and new code work simultaneously.
- + Set application graceful shutdown handlers to allow active requests to finish.
- + Use feature flags to separate code deployment from feature exposure.
- + Test rollbacks regularly to ensure you can revert quickly if an update fails.

Watch Out For

- ! Renaming or removing database columns in a single release, breaking active app instances.
- ! Forgetting readiness probes, causing users to hit app containers before they finish booting.
- ! Ignoring SIGTERM signals, causing ongoing user transactions to get killed abruptly.
- ! Deploying updates without proper health checks or automated rollback mechanisms.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh